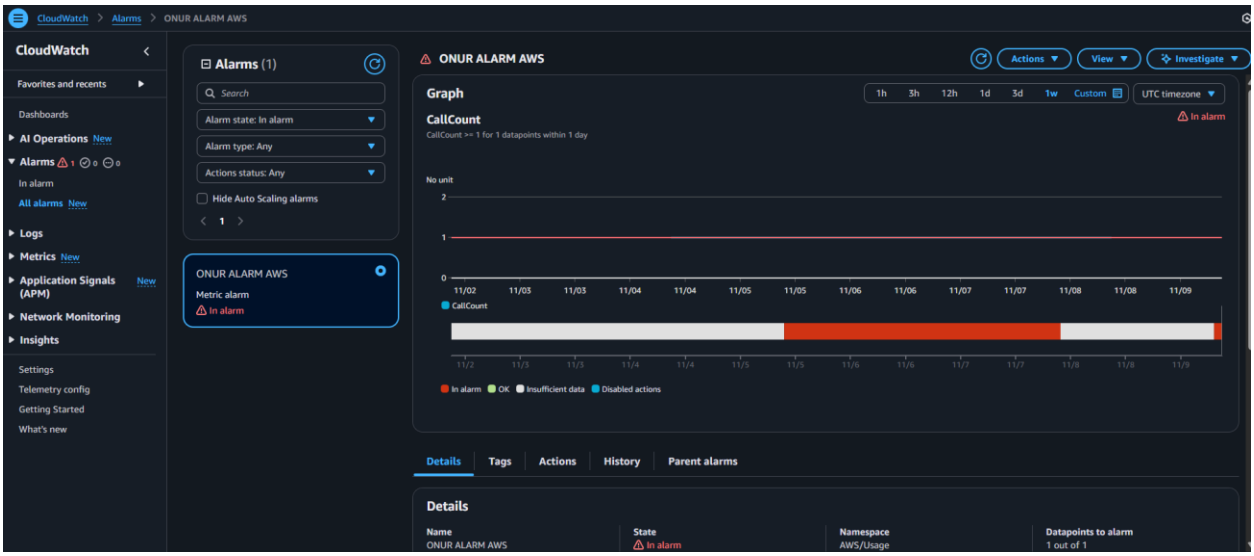
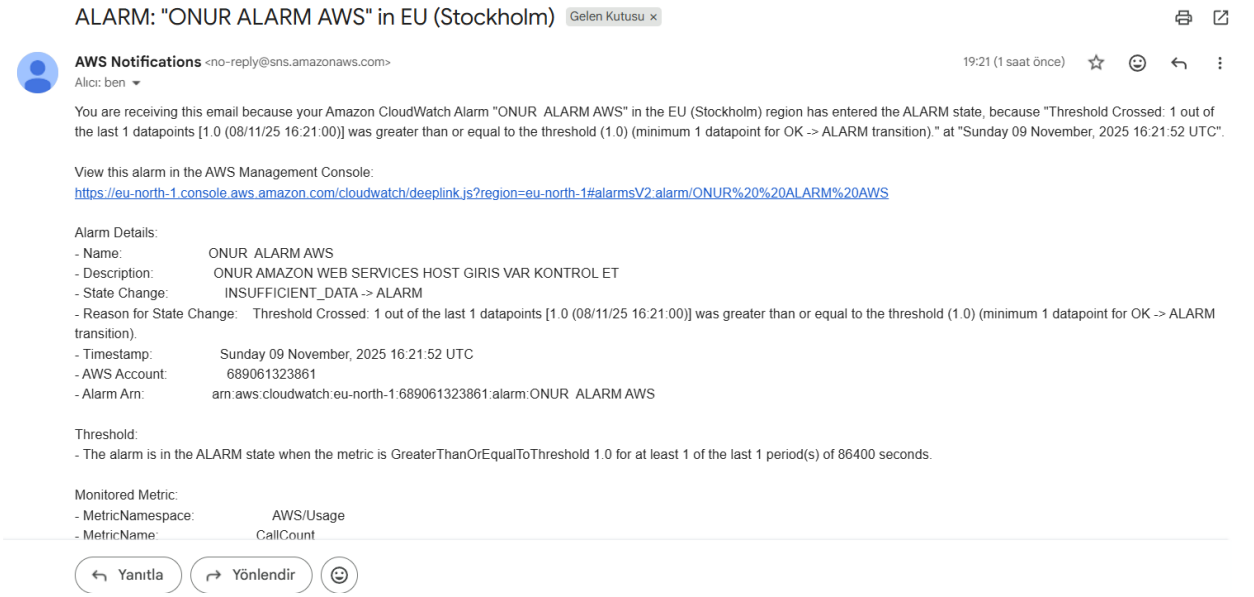
AWS Core Services Configuration Report

Report: AWS CloudWatch Alarm Setup and Triggering





Based on the screenshots, I have successfully set up a CloudWatch alarm in my AWS account in the EU (Stockholm) region and triggered this alarm.

Alarm Name: ONUR ALARM AWS

Description: ONUR AMAZON WEB SERVICES HOST LOGIN CHECK

Monitored Metric: The CallCount metric under the AWS/Usage namespace.

Status Summary: The alarm I created monitors the CallCount metric. The alarm's trigger condition is set to be equal to or greater than a value of 1.0.

The email I shared (first image) shows that the alarm status changed from INSUFFICIENT_DATA to ALARM. This occurred when the first data point received for the metric (1.0) matched or exceeded the threshold you set (1.0). The graph in the CloudWatch console (second image) also confirms this; the graph bar appears in red, indicating that the alarm has been triggered.

My Setup Steps

Here are the steps I took to achieve this result:

I Defined My Metric: First, I selected a metric to monitor in CloudWatch. In this example, I selected the CallCount metric under the AWS/Usage namespace.

I Started the CloudWatch Alarm: I went to the “Alarms” section in the AWS CloudWatch console and started the “Create alarm” wizard.

I configured the actions:

I specified what should happen when the alarm status changes to ALARM.

At this stage, I used the “Send notification” option and selected an Amazon SNS topic or created a new one.

I added my email address as a subscriber to this SNS topic and accepted the confirmation email I received (This allowed me to receive emails when the alarm was triggered).

I Named and Created the Alarm:

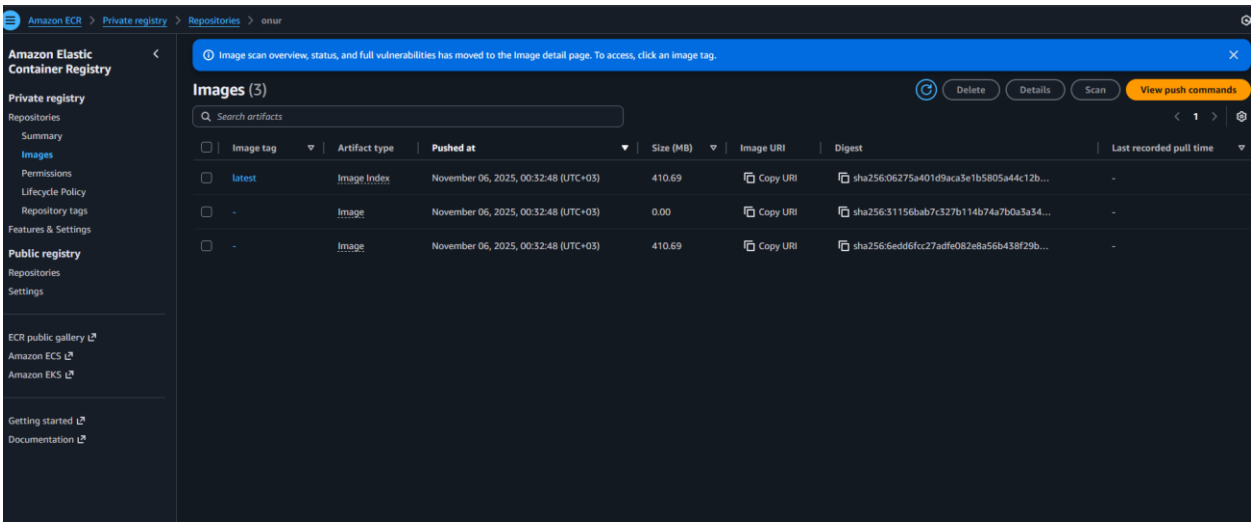
I entered a descriptive name for my alarm (ONUR ALARM AWS) and a description indicating what it monitors (ONUR AMAZON WEB SERVICES HOST LOGIN CHECK).

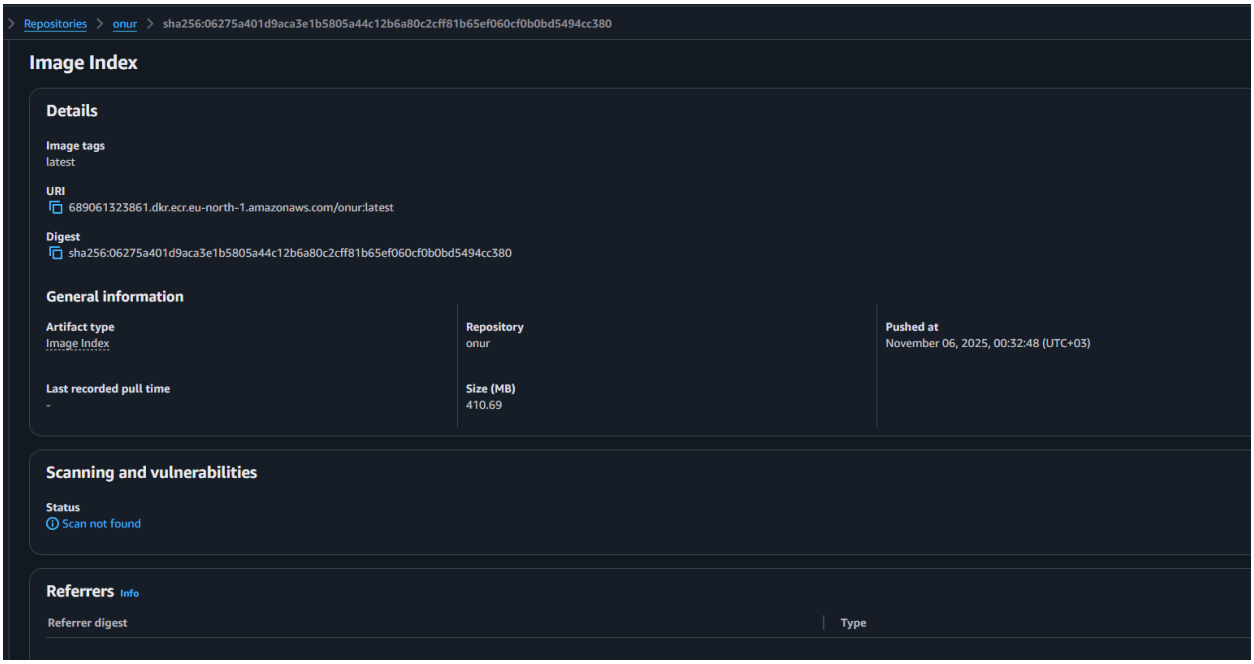
Verified the Trigger:

After setup, when the CallCount metric reported a value of 1.0, my alarm condition was met and the status changed to ALARM.

As a result, the SNS action was triggered, and I received the email notification shown in the first image.

Report: Pushing a Docker Image to an AWS ECR Private Repository





The screenshots I provided show that I used the Amazon Elastic Container Registry (ECR) service in my AWS account in the eu-north-1 (Stockholm) region. My goal was to push my local Docker image to a private ECR repository named onur.

As confirmed by the images, I successfully completed this process on November 6, 2025, and my image with the latest tag is currently stored in the repository.

My Repository Information (Taken from Images):

Repository Name: onur

Repository URI: 689061323861.dkr.ecr.eu-north-1.amazonaws.com/onur

Image Tag: latest

Region: eu-north-1

AWS Account ID: 689061323861

 Steps I Performed

To successfully register this image to ECR (or to repeat this process), I performed the following steps in order:

Preparation (AWS CLI and Docker): I made sure my local machine was configured with AWS CLI and Docker. I made sure my AWS CLI was configured with a profile that had access permissions (IAM permissions) to ECR.

Creating the ECR Repository:

I went to the ECR service in the AWS Management Console.

In the “Repositories” section and then “Create repository”.

I selected “Private” for visibility and entered onur as the repository name. I left the other settings as default and created the repository.

 Steps 2 ECR Authentication:

I ran an authentication command via the AWS CLI to allow Docker to access my ECR repository. This command obtained a temporary ECR login token for Docker.

 Steps 3 Local Image Tagging:

I had an image on my local machine that I built with docker build or pulled with docker pull (e.g., my-local-image:1.0).

To let Docker know which ECR repository to send this image to, I re-tagged it with my ECR Repository URI.

 Steps 4 Pushing the Image to ECR:

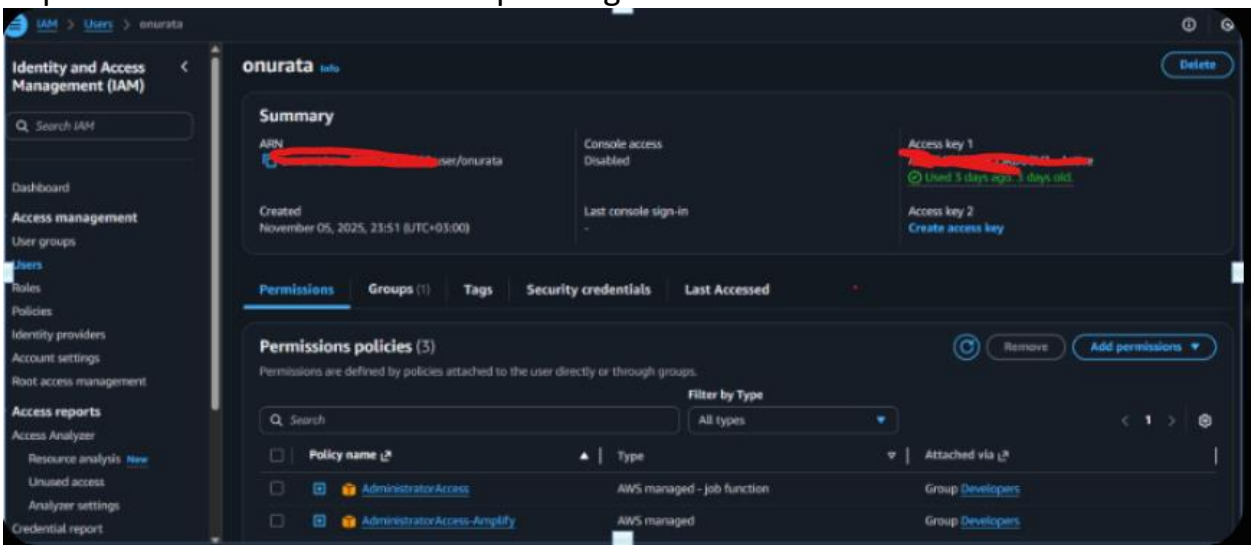
After tagging the image correctly, I pushed it to my onur repository in ECR using the docker push command.

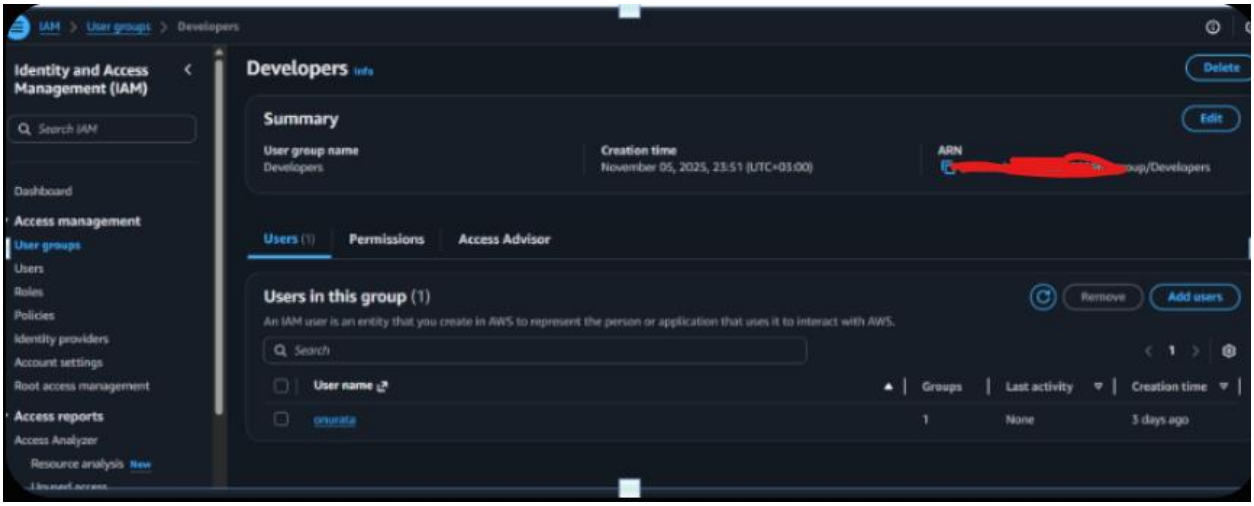
 Steps 5 Verification:

Finally, I returned to the ECR console and clicked on my onur repository.As in the first screenshot you sent, I verified that my image with the latest tag was successfully uploaded to the repository.As seen in the second image, the latest tag appears as an “Image Index”; this likely indicates that I grouped images for different platform architectures (e.g., arm64 and amd64) under a single tag.

At the end of this process, my Docker image was securely stored in ECR and was now ready for use in other AWS services such as Amazon ECS or EKS.

Report: AWS IAM User and Group Configuration





I configured the IAM service to securely manage my resources and services on AWS. My goal was to create a custom user to work via the command line (CLI) or APIs (programmatically) .

Summary of My Configuration (Taken from Screenshots):

User Name: onurata

User Group: Developers

Relationship: The onurata user is a member of the Developers group.

Permissions: Instead of assigning permissions directly to the onurata user, I assigned them to the Developers group. I added powerful (AWS-managed) policies such as AdministratorAccess and AdministratorAccess-Amplify to this group. This way, the onurata user inherited all these permissions held by the group.

Access Type: “Console access” is disabled for the onurata user. I created an Access key only for programmatic access, and this key is currently in the “Active” state. I used this key when installing the AWS CLI on my local machine.

Steps I Took to Set Up

I followed these steps to achieve this configuration:

I Created a User Group: First, I went to the IAM service and created a new group called Developers under the “User groups” section.

Added Policies to the Group: While creating the group, I specified the permissions that users belonging to this group would need. Since I needed broad permissions for my work, I selected policies such as AdministratorAccess managed by AWS and added them to this group.

Created an IAM User: I went to the “Users” section and created a new user named onurata.

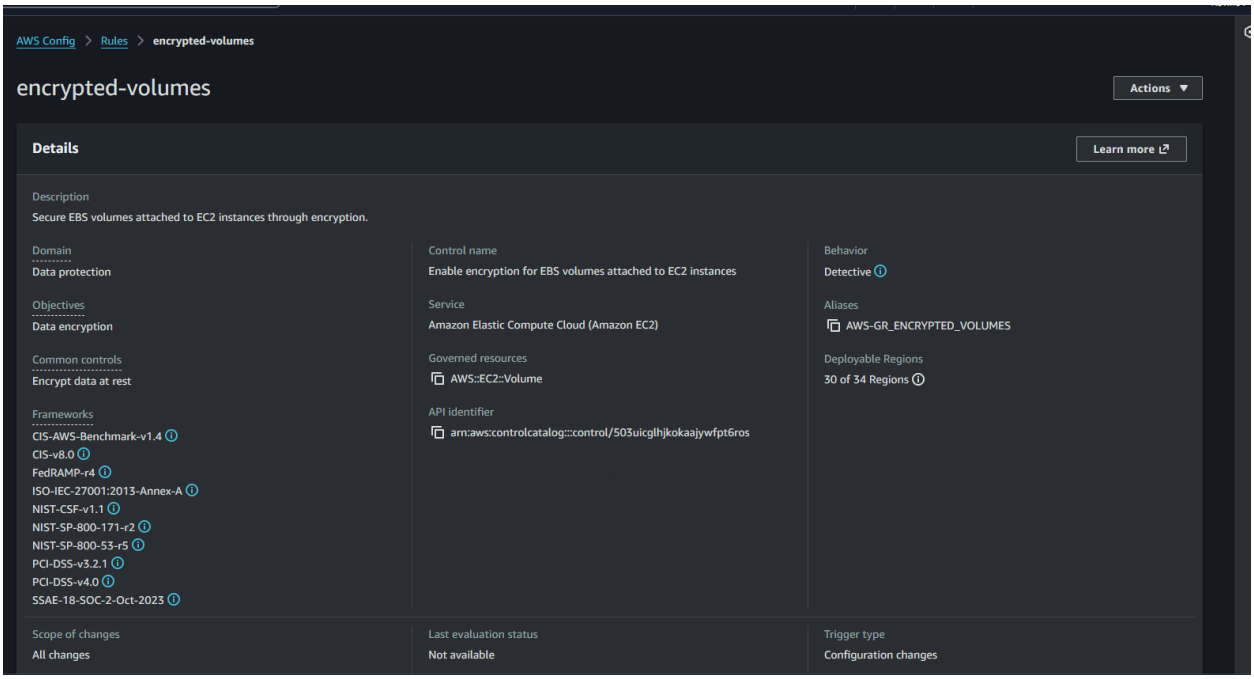
Access Type: When creating the user, I only selected the “Programmatic access” option as the access type. I intentionally disabled the “Console access” option.

Added the User to a Group: In the “Permissions” step of the user creation wizard, I selected the “Add user to group” option and selected the Developers group I had created earlier.

Access Keys: At the end of the wizard, AWS showed me an Access Key ID and Secret Access Key for the user. I saved this information in a secure location.

Verification (AWS CLI): I entered the Access Key ID and Secret Access Key information I just obtained by running the aws configure command in my local terminal. This allowed me to securely perform operations such as uploading images to ECR (docker push) and creating CloudWatch alarms through my terminal, as previously reported.

Report: EBS Volume Encryption Compliance Check with AWS Config



To strengthen the security and compliance posture of my AWS account, I used the AWS Config service this time. In my previous steps, I managed users with IAM, images with ECR, and alarms with CloudWatch; now I wanted to check whether my resources comply with my defined security standards.

In this setup, I did not create a new volume. Instead, I enabled a rule that automatically checks whether my existing or future EBS (Elastic Block Store) volumes are encrypted.

Summary of My Configuration:

Service Used: AWS Config

Enabled Rule: encrypted-volumes (Also Known As: AWS-GR_ENCRYPTED_VOLUMES)

Rule Purpose: “Ensure that EBS volumes attached to EC2 instances (virtual servers) are secured through encryption.”

Rule Behavior: “Detective”

The “Detective” behavior means that this rule does not automatically fix an issue (such as an unencrypted disk), but instead detects it and reports it to me as “Non-compliant.” This allows me to be immediately notified of security breaches and intervene manually.

Steps I Took to Set Up:

AWS Config Service: I opened the “AWS Config” service in the AWS Management Console.

Launched the Add Rule Wizard: I went to the “Rules” section in the console and clicked the “Add rule” button.

Selected a Predefined Rule: I opened the list of predefined managed rules offered by AWS.

Found and Selected the Rule: I searched the list for rules related to “encryption” and found and selected the encrypted-volumes rule (or the rule with the alias AWS-GR_ENCRYPTED_VOLUMES) shown in the screenshot. I saw that this rule is also used for many security standards such as CIS and PCI-DSS.

The audit: After the rule was enabled, AWS Config began scanning all AWS::EC2::Volume resources in my account. Now, if I create an unencrypted EBS volume or if one of my existing volumes is unencrypted, I will see this volume marked as “Non-compliant” in the AWS Config dashboard and will need to take corrective action (manually encrypt the volume).